

ExTernD at Ternary Bit-Rates: Can Rank-Expanded Ternary Decomposition Reach 1.52–2.06 bpw?

TL;DR

No. ExTernD's representation cannot reach 1.52/1.71 bpw at quality $\geq$ a single per-group-scaled ternary plane, and only barely reaches ~ 2.06 bpw on the least-rectangular MLPs. The iso-bpw math forces the inner-rank multiplier μ **below 1 (rank-deficient)** for every projection at ternary bit-rates — a regime where the paper's own crossover data ($\mu \approx 1.5$ already loses to 2-plane PTQTP by 1.8–2.6 energy points) extrapolates to catastrophic loss versus the ~ 78 – 81% energy of one dense plane.

What is worth salvaging is not the representation but three pieces of machinery: (i) a *tiny- μ , imatrix-weighted residual correction* deployed as a runtime LoRA adapter (no graph patch, but violates "no side tensors" and pushes you over 2.06 bpw); (ii) the *decorrelated block-ALS + joint-scale solve* as a drop-in optimizer for your existing TQ2_0 assignment search; (iii) the *plain-residual V-step* insight for importance weighting, directly relevant to your Gated-DeltaNet contractivity-protection projections.

What actually moves quality at 1.6–2.1 bpw within scalar ternary is assignment optimization (GPTQ/OBS-on-the-grid, à la PT2-LLM's ITF+AGA, plus the Hadamard rotation you already use) and QAT (BitNet-class). Beating scalar ternary at fixed bpw *requires abandoning scalar ternary* for VQ/trellis (QTIP/QuIP#/GLVQ) or lattice IQ-quants — none of which are native to TQ2_0.

Key Findings

The 1.52/1.71 targets are single-plane entropy points, not multi-factor budgets. $\log_2(3) = 1.585$ bpw is the max entropy of a dense uniform trit. Your $1.71 \approx 1.585 + 0.125$ (dense trit + 8-bit group-64 scale, since $8/64 = 0.125$); your $1.52 \approx$ the Shannon entropy of a ~ 48 – 50% -sparse single ternary plane (entropy = $H(s) + (1-s) \cdot H(0.48) + 0.52 = 1.519$; $H(0.50) + 0.50 = 1.500$). llama.cpp TQ1_0 = 1.6875 bpw (5 trits/byte, since $3^5 = 243 < 256 = 2^8$; compile PR #8151) and TQ2_0 = 2.0625 bpw (2 bits + f16 block scale). These are all **one-plane** budgets. ExTernD spends its bits on *two* ternary factors whose element count is $\mu \cdot (m+n) / \max(m,n) \times$ the original — inherently a multi-bit representation.

Iso-bpw is sub-unitary everywhere at ternary rates. Using the paper's cost model $\text{bpw}_{\text{eff}} =$

... $\mu \cdot (m+n) / \max(m,n) \cdot b_{\text{elem}}$ (tables below), hitting 1.52 or 1.71 bpw requires $\mu \approx 0.4\text{--}0.96$ across all realistic aspect ratios and all three storage costings. $\mu < 1$ means $k < \min(m,n)$: a **rank-deficient** ternary factorization — a regime ExTernD never tests (its experiments use $\mu = 1.97\text{--}7.21$) and one where a flat/Marchenko–Pastur weight spectrum caps energy at roughly the retained-rank fraction μ *before* the double-ternarization penalty.

The rate-distortion ceiling explains why. For a Gaussian source, $D(R) = \sigma^2 \cdot 2^{(-2R)}$; at $R = 1.585$, $D = 2^{(-3.17)} = 0.111$, so **88.9% energy is the hard ceiling for any quantizer at 1.585 bits** — including VQ. The MSE-optimal 3-level scalar quantizer for a Gaussian (threshold $\alpha^* = \sigma \sqrt{2} \cdot \text{erfinv}(2/3) \approx 0.798\sigma$) leaves ~ 0.19 normalized MSE $\rightarrow$ **$\sim 81\%$ energy**, matching PT2-LLM's measured 78–81%. Per-group scaling recovers the non-stationarity gap (why TQ2_0's per-g64/g128 scales beat a single global scale). The ~ 8 -point gap between 81% (scalar ternary) and 88.9% (R-D bound) is capturable only by high-dimensional VQ/trellis, **not** by rank expansion, which spends bits on more ternary DOF rather than better-shaped codewords. [arxiv](#)

ExTernD is uncompetitive even at its home bit-rate. The full Qwen3.5-4B conversion at ~ 5.7 effective bpw scores WikiText-2 PPL 10.099 vs bf16 9.782 (+3.2%), and the paper's own end-to-end table shows it trailing pure Q5_K by ~ 1.5 points of relative PPL at matched bits. If rank-expanded ternary loses to a plain 5-bit k-quant at 5.7 bpw, a rank-deficient version at 1.7 bpw is hopeless. [arXiv](#)

Details

1. Iso-bpw math

Aspect ratios $r = (m+n) / \max(m,n)$, computed from actual Qwen3-30B-A3B (`[Qwen3MoeForCausalLM]`: hidden 2048, head_dim 128, 32 Q / 4 KV heads, expert intermediate 768, 48 layers) and Qwen3.6-35B-A3B (`[qwen3_5_moe]` hybrid: hidden 2048, head_dim 256, 16 Q / 2 KV heads, 256 experts, expert intermediate 512, 40 layers = 30 Gated-DeltaNet + 10 full-attention):

Projection	Example dims (m×n) r = (m+n)/max	
Square (idealized)	n×n	2.000
q_proj / o_proj (Qwen3-30B)	4096×2048	1.500
MoE expert gate/up/down (Qwen3-30B)	768×2048	1.375

GQA k/v (Qwen3-30B)	512×2048	1.250
MoE expert / GQA k/v (Qwen3.6-35B)	512×2048	1.250
Gemma 8:1 MLP (paper)	12288×1536	1.125

Per-factor-element storage cost b_{elem} for the three costings:

- (a) **mask+sign** = $2-s$: $\tau=0.7$ ($s=0.41$) $\rightarrow$ 1.59; $\tau=1.0$ ($s=0.57$) $\rightarrow$ 1.43; $\tau=2.0$ ($s=0.87$) $\rightarrow$ 1.13.
- (b) **TQ2_0** = 2.0625 regardless of s .
- (c) **entropy** = $H(s)+(1-s)$: $\tau=0.7\rightarrow$ 1.566; $\tau=1.0\rightarrow$ 1.416; $\tau=2.0\rightarrow$ 0.687.

Solving $\mu = B / (r \cdot b_{\text{elem}})$:

Table A — μ required, mask+sign packing at $\tau=0.7$ ($b_{\text{elem}}=1.59$):

r	$B=1.52$	$B=1.71$	$B=2.06$
2.000	0.48	0.54	0.65
1.500	0.64	0.72	0.86
1.375	0.70	0.78	0.94
1.250	0.77	0.86	1.04
1.125	0.85	0.96	1.15

Table B — μ required, TQ2_0 storage ($b_{\text{elem}}=2.0625$):

r	$B=1.52$	$B=1.71$	$B=2.06$
2.000	0.37	0.41	0.50
1.500	0.49	0.55	0.67
1.375	0.54	0.60	0.73
1.250	0.59	0.66	0.80

1.125 0.66 0.74 0.89

Table C — μ required, entropy coding at $\tau=1.0$ ($b_{\text{elem}}=1.416$, the paper’s mask+sign interior optimum):

r	B=1.52	B=1.71	B=2.06
2.000	0.54	0.60	0.73
1.500	0.72	0.81	0.97
1.375	0.78	0.88	1.06
1.250	0.86	0.97	1.16
1.125	0.95	1.07	1.29

Reading the tables: every cell at $B=1.52$ and $B=1.71$ is $\mu < 1$ (rank-deficient). Only $B=2.06$ on the least-rectangular MLPs ($r \leq 1.25$) and with mask+sign/entropy coding reaches $\mu \approx 1.0$ – 1.3 — precisely the $\mu \in [1, 1.5]$ band the paper shows already losing to dense planes (its $\mu \approx 1.5$, element-matched, trails PTQTP $K=2$ by 1.8–2.6 energy points). The very rectangular GQA k/v projections ($r=1.25$) and square DeltaNet projections ($r=2.0$) are the *worst* case: they force the deepest rank deficiency for a given bpw. (Note: $r=1.25$ appears both for GQA k/v and Qwen3.6 experts, but for opposite reasons — the k/v case is rectangular, the expert case is near-square-narrow; both still force $\mu < 1$.)

2. Rank-deficient ternary factorization viability

LLM weight matrices are close to full-rank. Measured spectra (SVD-LLM on LLaMA-7B; the LLaMA-2-7B per-layer plots in the gradient-subspace-stabilization work; the Chen et al. “PLM weights are not inherently low-rank” result) show flat, Marchenko–Pastur-like bulk with only a handful of large singular values — which is exactly why SVD-LLM/ASVD must whiten with activation statistics and still lose accuracy at high compression, and why LoRD/low-rank methods need rank in the hundreds. For a flat spectrum, a rank- $r = \mu \cdot \min(m, n)$ real-valued approximation captures $\approx \mu$ of the energy; **ternarizing both factors strictly loses more**. So at $\mu \approx 0.5$ – 0.9 you are structurally below the $\sim 81\%$ a single full-rank-structured ternary plane achieves.

The binary/ternary factorization literature confirms the direction: OneBit (SVD-based value/sign decomposition, QAT) reaches ~81–90% of FP on LLaMA only *with* distillation; ARB-LLM (alternating refined binarization + residual, ~1.1 bit) needs residual planes and grouping and still lands at WikiText2 ~14 on LLaMA-7B; only BTC-LLM's learnable transform + binary codebook brings 1.1-bit to PPL 6.23; STBLLM breaks 1 bit only with N:M sparsity and heavy accuracy loss. None of these are pure rank-deficient ternary factorizations, and all that approach usable quality add either residual planes or codebooks — i.e., they abandon the single-low-rank-ternary-product form. **Verdict: $\mu < 1$ ExTernD cannot beat the ~81% energy of a single per-group-scaled plane at equal bpw.** The paper's own $\mu \approx 1.5$ crossover, extrapolated downward, makes this unambiguous. [arxiv + 2](#)

↗

3. Rate-distortion ceiling and where the 2-bit VQ/trellis methods sit

Gaussian R-D bound at 1.585 bits: 88.9% energy ceiling (any quantizer).

Optimal scalar ternary: ~81% energy ($\alpha^* \approx 0.798\sigma$), $\approx$ PT2-LLM's 78–81%.

The 2-bit VQ/trellis family closes most of the residual gap by shaping in high dimension: on LLaMA-2-70B (fp16 WikiText2 = 3.12), QuIP# 2-bit = 3.91, QTIP = 3.78, GLVQ-32D = 3.36 (source: "Learning Grouped Lattice Vector Quantizers," arXiv:2510.20984, Table 1); GPTVQ-2D 2-bit LLaMA-7B = 9.64. They use E8-lattice (QuIP#), 8D codebooks (AQLM), or trellis codes (QTIP) — **codebook/trellis lookups, not rank expansion**. This is the empirical proof that at ~2 bpw the winning move is better codewords, not more ternary factors. [arxiv](#)

↗

llama.cpp support (from the official llama-quantize help): IQ2_XXS = 2.06 bpw, IQ2_XS = 2.31, IQ2_S = 2.5, IQ2_M = 2.7, IQ1_S = 1.56, IQ1_M = 1.75 — all E8-lattice-style codebooks (ikawrakow). TQ1_0 = 1.69 and TQ2_0 = 2.06 are ternary-model-only (compilade, PR #8151) and dequantize to int8 for dp4a kernels. On a Qwen3-class 27–35B model, an independent KLD benchmark found UD-IQ2_XXS at ~1.13 $\times$ baseline perplexity and *every* IQ1 candidate strictly worse — a useful real-world anchor for what sub-2-bit costs on your architecture. The IQ2 lattice quants are the strongest thing llama.cpp actually ships in your bpw window, and they beat scalar-ternary quality at equal bits — but they are not TQ2_0 and not multiplication-free. [Hugging Face](#)

↗

4. Hybrid: single plane + tiny- μ ternary residual

Form $A \approx S \odot T + B \cdot \text{diag}(D) \cdot C$, with $S \odot T$ the TQ2_0 plane and B,C a small- μ_r ExTernD fit to the (optionally imatrix-weighted) residual. This is the LQER / ZeroQuant-V2 LoRC / CALDERA / QLoRA-error-reconstruction family.

What the literature measures: LQER needs rank 254 for usable 2-bit. FLQR brings the effective

what the literature measures: LQER needs rank 256 for usable 2-bit; FLERQ brings the effective rank to ~40 with flexible selection; CALDERA's best 2-bit results use rank 256 with 4-bit factors (and pays large inference latency for it). The reason rank must be high: **the quantization residual of a good RTN/GPTQ/ternary plane is near-white (high-rank)**, so a rank- r Frobenius correction captures only $\approx r/\min(m,n)$ of residual energy — *unless* the residual is concentrated in a few important input directions, which is exactly what activation/imatrix weighting exploits (LQER's L^2 QER left-multiplies the error by an activation-magnitude scale matrix before SVD; this is why it works at all). [arxiv + 2](#)

↗

This is directly relevant to your Gated-DeltaNet gate/decay projections. The paper reports 4.5–7.4% μ reduction from imatrix weighting on gated-DeltaNet qkv/out projections, i.e., their input importance is strongly skewed — the one place a rank- r correction is efficient. These are also the projections you must protect for contractivity (spectral norm ≤ 1). A *very* small imatrix-weighted residual there could buy disproportionate quality.

bpw cost. Residual adds $\mu_r \cdot r \cdot b_{\text{elem}}$ bpw. For $\mu_r = 0.0625$ (rank $\approx 6\%$ of min-dim), $r = 1.375$, TQ2_0 storage: $0.0625 \cdot 1.375 \cdot 2.0625 = \mathbf{0.18 \text{ bpw}}$. On top of a TQ2_0 base plane (2.0625) that is 2.24 bpw — already over your 2.06 ceiling. Only a TQ1_0 base (1.6875) + tiny residual lands in the 1.85–2.06 window. Under mask+sign at high sparsity the residual is cheaper ($0.0625 \cdot 1.375 \cdot 1.13 = 0.097$ bpw) but needs a sparse kernel llama.cpp lacks.

Deployability without a graph patch. llama.cpp supports runtime `--lora` adapters applied as $y = Wx + B(Ax)$ *without merging* (ngxson's server LoRA, PR #8332; `llama_adapter_lora_init` / `llama_set_adapter_lora`), and `convert_lora_to_gguf.py` can quantize adapters (e.g., `--quantize q8_0`). So the hybrid's residual **can** be shipped as a standard low-rank LoRA adapter with *no* model-graph change — this is the single most practical way to relax your "no side tensors" constraint cheaply. Caveats: (i) llama.cpp LoRA adapters are conventional dense low-rank tensors at f16/q8, **not** ternary ExTernD factors — you would store the residual as ordinary A,B low-rank matrices, forfeiting the "everything ternary" property; (ii) adapters live in host/GPU memory as extra tensors, violating "no side tensors"; (iii) applying at runtime is three matmuls, a decode-time cost. Storing genuinely *ternary* TQ2_0 adapter tensors is not supported and would require kernel/loader work. [huggingface](#)

↗

[LobeHub](#)

↗

5. Shared-factor savings

For SwiGLU gate/up sharing input-side C ($k \times n_{\text{hid}}$) with separate B gate.B up: savings = $n_{\text{hid}} /$

$[2(n_{\text{int}}+n_{\text{hid}})]$.

Gemma-style wide MLP ($n_{\text{int}}=12288$, $n_{\text{hid}}=1536$): **5.6%**.

Qwen3-30B narrow MoE expert ($n_{\text{int}}=768$, $n_{\text{hid}}=2048$): **36.4%** (large because experts are narrower than hidden).

Attention q/k/v sharing input C saves more when fused, but GQA makes k/v tiny (512 wide) so the absolute win is modest.

Even the 36% expert-case saving multiplies μ by ~ 0.64 , nowhere near the $\sim 3\times$ reduction needed to go from a competitive $\mu \approx 2.5$ to the $\mu \approx 0.6\text{--}0.9$ that ternary bpw demands. **Shared-C does not change the iso-bpw verdict.** It also risks interacting badly with your finding that SwiGLU gate/up are multiplicatively coupled: a shared C forces both branches to read the same ternary projection of x , collapsing the pre-activation subspaces the $\text{SiLU}(\text{gate}) \odot \text{up}$ product relies on. Treat shared-C as an optimizer-side memory optimization at high μ , not a route to ternary bpw.

6. High- τ sparsity + entropy coding

At $\tau=2.0$ ($s=0.87$), ideal entropy = $H(0.13)+0.13 \approx 0.687$ bits/element. But the paper's own nonzero budget $\mu \times \text{density}$ is roughly conserved across τ (falls only $1.65 \rightarrow 0.87$ from $\tau=0.7 \rightarrow 2.0$), and required μ for 99% energy on up_proj rises to 7.21. Iso-bpw: $7.21 \cdot 1.125 \cdot 0.687 \approx \mathbf{5.6 \text{ bpw}}$ — essentially unchanged from the other τ points. There is **no (τ, μ) point that lands under ~ 2 bpw at energy comparable to ($\sim 80\text{--}85\%$) or better than a single plane**: to get under 2 bpw at high τ you must drop μ back below 1, re-entering the rank-deficient failure regime.

llama.cpp has no sparse or entropy-coded weight format — all quant types are dense fixed-bpw blocks (confirmed across TQ/IQ/Q_K families). An unstructured-sparse ternary kernel on a 3090 is a poor fit: irregular gathers defeat dp4a/tensor-core throughput; 2:4 structured sparsity gives only 50% and would not match ExTernD's 87% *unstructured* supports. Entropy coding is a storage-only trick with no matching CUDA matmul path. So the entropy-floor bpw numbers in the paper are not realizable as speed on your hardware.

7. ExTernD machinery as a training-time device

Collapsing $B \cdot \text{diag}(D) \cdot C$ back to one per-group-scaled plane is just re-quantizing the product $\hat{A}$ onto the ternary grid — it **discards the extra capacity** and is a no-op-or-worse in Frobenius versus quantizing A directly (projecting the smoothed $\hat{A}$ can even move mass off good grid points). There is no strong evidence in the over-complete-representation literature (Bop/latent-weight BNN methods, ternary dictionary learning) that expansion-then-projection yields better

single-plane assignments than direct STE/GPTQ on the grid. The paper itself flags the open question of whether STE gradients behave through *two chained ternary factors*; the BNN-with-factorized-layers experience (XNOR-Net factorization, ternary Kronecker) is that chained low-precision products increase gradient variance and are hard to train — consistent with your own finding that assignment training via STE latents saturates. **This route is unpromising**; your observed fixed-assignment ceiling is a property of the ternary grid's capacity, and adding a *transient* rank-expanded parameterization that must be re-collapsed does not raise that ceiling.

8. Related / follow-up work (bpw / quality landscape)

Method	Class	bpw	Quality (WikiText2 PPL unless noted)	llama.cpp?
BitNet b1.58 2B4T	native ternary QAT	~1.58 core	GSM8K 58.38, MMLU 53.17, ARC-C 49.91, WinoGrande 71.90, CSQA 71.58 (on par with similar-size FP)	custom W1.58A8 kernel; TQ1_0/TQ2_0 in GGUF
Ternary Bonsai (1.7B/4B/8B)	native ternary QAT	~1.58	vendor claims "top intelligence at 1.58 bits"; your quality target	needs group-128 ternary (open discussion #22019)
PT2-LLM	single-plane PTQ	1.58	LLaMA-7B 11.39 / C4 24.55; zero-shot avg 45.07 (FP16 5.68 / 61.73)	no native format
PTQTP	K stacked trit-planes	K=2 ≈ 3.2- 3.4	recovers ~94% of PPL gap; near-QAT; 82.4% math-reasoning retention	no
TWLA	ternary PTQ + 4-bit act	1.58 (W1.58A4)	beats 2-bit GPTQ/QuaRot/SliM-LLM	no
Tequila	trapping-free ternary QAT	~1.58	improves over BitNet-class	no
OuIP#	E8-lattice VO PTO	2.0	LLaMA-2-70B 3.91	≈ IQ2 lattice analogue

QTIP	trellis PTQ	2.0	LLaMA-2-70B 3.78	no (custom kernels)
GLVQ-32D	grouped lattice VQ	2.0	LLaMA-2-70B 3.36 (fp16 3.12)	no
GPTVQ-2D	vector-quant PTQ	2.0	LLaMA-7B 9.64	no
llama.cpp IQ2_XXS	E8-style lattice	2.06	~1.13× baseline PPL on Qwen3-27B (KLD test)	yes
llama.cpp IQ1_M / IQ1_S	lattice	1.75 / 1.56	worse than IQ2 on Qwen3-27B	yes
ExTernD (full 4B)	rank-expanded ternary	~5.7	Qwen3.5-4B 10.099 (bf16 9.782); trails Q5_K	stores factors in TQ2_0

No public GitHub release for ExTernD / "expanded-rank ternary" was found, and no llama.cpp issue/PR references it; treat all extrapolations to $\mu < 1$ as *unmeasured*. The clear pattern: the methods that beat scalar per-group ternary at equal bpw are all either (a) native QAT (BitNet/Bonsai/Tequila), or (b) VQ/trellis/lattice (QuIP#/QTIP/GLVQ/IQ2). No post-training rank-expansion method beats a plane at ternary bpw.

9. Verdict and ranked recommendations

(a) Representation verdict. ExTernD as a representation **cannot** reach 1.52 or 1.71 bpw at quality $\geq$ a single per-group-scaled ternary plane, and only marginally reaches 2.06 bpw on the least-rectangular MLPs ($\mu \approx 1.0\text{--}1.3$), where the paper's crossover data already shows it losing to a 2-plane baseline. The reasons are structural and stack: (i) ternary bpw forces $\mu < 1$, a rank-deficient regime; (ii) LLM weights are near-full-rank, so rank-deficiency caps energy below a plane before ternarization; (iii) the 88.9% Gaussian R-D ceiling at 1.585 bits is capturable by VQ shaping, not by more ternary DOF; (iv) ExTernD trails a plain 5-bit k-quant even at 5.7 bpw.

(b) Machinery worth your time (ranked):

Decorrelated block AFS + joint codebook co-assignment optimizer (within constraints)

Decorrelated block-ALS + joint-scale solve as an assignment optimizer (within constraints).

The insight that ternarizing the *jointly decorrelated* least-squares target — $V \leftarrow T_{\tau}((U^T U + \epsilon I)^{-1} U^T R)^T$ — avoids the column-wise collapse (99.5% → 50%) is a general lesson for any block-coordinate ternary assignment search, including single-plane. **bpw cost: 0. Expected gain: modest but free** — a better local optimum on the fixed TQ2_0 grid, complementary to GPTQ-style compensation. Does **not** violate any constraint.

imatrix-weighted normal equations with a *plain-residual* V-step (within constraints). The paper's finding that the U-step/scale solve should use weighted normal equations but the V-step must use the *unweighted* residual R (weighting the V-target costs 0.3–1.8 points) is a concrete, transferable recipe. Apply it to your imatrix-driven scale/assignment search, **especially on the Gated-DeltaNet gate/decay projections** where importance is most skewed and contractivity must be preserved. **bpw cost: 0. Expected gain: small-to-moderate on the skewed projections.** Within constraints.

Tiny- μ imatrix-weighted residual as a runtime LoRA adapter (relaxed constraint). Only worth it if you relax "no side tensors." Fit a rank- r ($r \approx 16$ –64) *activation-weighted* correction to the TQ1_0/TQ2_0 residual on the contractivity-critical DeltaNet projections and a few worst MLP down_projs, ship as a quantized `--lora` adapter (no graph patch). **bpw cost: ~0.1–0.2 bpw per corrected tensor; pushes a TQ2_0 base over 2.06, so pair with a TQ1_0 base to stay ≤ 2.06 .** **Expected gain: LQER-class — meaningful only where the residual is directionally concentrated; near-useless on white-residual MLPs.** Violates "no side tensors"; store as *dense* low-rank (not ternary) since llama.cpp has no ternary adapter path.

Shared-C — skip as a bpw lever (only 6–36% factor savings, doesn't reach 1.7 bpw, and risks the SwiGLU coupling). Consider only as an optimizer-memory optimization at high μ .

(c) What the literature says actually moves quality at 1.6–2.1 bpw:

Within scalar ternary (your constraints): assignment optimization on the grid (GPTQ/OBS-style error compensation; PT2-LLM's Iterative Ternary Fitting + Activation-aware Grid Alignment + structural reordering — ITF alone cut LLaMA-2-7B WikiText2 from 22.88 to 15.47, AGA lifted zero-shot avg from 37.11 to 42.86), per-group scale search, imatrix weighting, and Hadamard/QuaRot rotation (which you already use — it Gaussianizes the block, making $\alpha^* \approx 0.798\sigma$ near-optimal and is the single highest-leverage preprocessing at ternary rates). QAT (BitNet-class) is the ceiling and is what Ternary Bonsai — your quality target — uses.

Requires abandoning scalar ternary: VQ/trellis/lattice codebooks (QuIP#, QTIP, GLVQ, AQLM) and llama.cpp's IQ2_XXS/IQ2_XS/IQ2_M and IQ1_M/IQ1_S lattice quant. On your Qwen3-class

model these are the only *shipping* option that beats scalar-ternary quality at ~2 bpw, but they are not TQ2_0, not multiplication-free, and use codebook lookups.

are not TQ2_0, not multiplication-free, and use codebook lookups.

Decision: Do not pursue ExTernD as a ternary-bpw representation. Port its block-ALS/joint-scale and plain-residual-V-step *ideas* into your existing TQ2_0 assignment+scale search (zero bpw, within constraints). If — and only if — your fixed-assignment ceiling on the DeltaNet projections remains the binding quality limit after those, pilot a small runtime-LoRA residual on just those projections and benchmark against simply switching those tensors to IQ2_XXS.

Recommendations

Immediately (0 bpw, in-constraint): Add the decorrelated joint least-squares target and joint diagonal-scale solve $((U^T U) \odot (V^T V))d = \text{diag}(U^T R V)$ as an inner optimizer step in your single-plane TQ2_0 assignment search; use imatrix-weighted normal equations for scales but keep the assignment (V-step) target unweighted. Measure WikiText-2 PPL and per-tensor energy delta versus your current search.

Next (0 bpw, in-constraint): Confirm Hadamard/QuaRot rotation is applied per-g64/g128 block before ternarization on all projections; verify the empirical block std drives $\alpha \approx 0.798 \cdot \sigma$. This is the highest-leverage scalar-ternary move and directly raises the ~81% single-plane energy toward the 88.9% ceiling on Gaussianized blocks.

Conditional pilot (relaxed constraint, +0.1–0.2 bpw): Only on the Gated-DeltaNet gate/decay/qkv/out projections (skewed importance + contractivity-critical), fit a rank-16–64 activation-weighted residual and deploy as a quantized runtime `--lora` adapter over a TQ1_0 base. Benchmark head-to-head against setting exactly those tensors to IQ2_XXS. **Escalation threshold:** justify the pilot only if, after steps 1–2, the DeltaNet-protected build's WikiText-2 PPL gap to bf16 has plateaued above your tolerance; abandon it if IQ2_XXS on those tensors matches or beats it at equal-or-lower total bpw.

Do not pursue $\mu < 1$ ExTernD, shared-C for bpw reduction, high- τ entropy coding (no kernel), or expansion-then-recollapse as an assignment initializer.

Benchmarks that change the plan: if Ternary Bonsai-class QAT becomes feasible in your pipeline, it supersedes all PTQ options at 1.58 bpw; if llama.cpp lands a group-128 ternary format (discussion #22019, which notes TQ1_0/TQ2_0's group-256 is non-native for group-128 models) it changes your scale-granularity tradeoff; if an IQ2 lattice build on the full 27–35B fits your 2×24 GB budget at acceptable PPL, it is the pragmatic sub-2-bit answer.

Caveats

Extrapolations vs measured: All $\mu < 1$ energy claims are *extrapolations* from the paper's $\mu \geq 1.5$ data and from flat-spectrum/R-D theory: ExTernD publishes no rank-deficient results. The iso-

bpw μ values in Tables A–C are exact given the paper's cost model but assume its sparsity-vs- τ relationship holds at low μ (untested).

Aspect ratios: Qwen3.6-35B-A3B dims were read from the HF config blob, not raw JSON; the `attn_output_gate=true` flag may alter `q_proj` output width (possibly doubling to 8192, lowering r to ~ 1.25) — treat that projection's r as uncertain. Qwen3-30B-A3B dims are confirmed. Both models use hidden 2048 with narrow experts (768 / 512), so their MLP aspect ratios (1.25–1.375) are far less rectangular than the Gemma 8:1 MLPs in the paper ($r=1.125$), meaning the paper's *best-case* MLP numbers do not transfer to your architecture.

R-D numbers assume a Gaussian source; real weight blocks post-Hadamard are near-Gaussian but not exactly, so 88.9%/81% are close approximations, not guarantees. The optimal-ternary $\sim 81\%$ figure aligns with PT2-LLM's measured 78–81%, which lends confidence.

PTQTP bpw is reported inconsistently (marketed as "1.58-bit" via a 2×1.58 -bit representation but structurally 2 stacked planes ≈ 3.2 – 3.4 bpw); the ExTernD paper's own accounting ($K=2 \approx 3.4$ bpw) is used here.

Runtime-LoRA quantized-adapter path is confirmed to exist for conventional dense low-rank adapters (q8_0), but ternary/TQ2_0 adapter tensors are **not** supported; the hybrid would ship dense low-rank factors, not ExTernD ternary factors.

Perplexity-only: ExTernD reports no downstream/KLD; its $\sim +3.2\%$ PPL at 5.7 bpw may understate task degradation. BitNet/Bonsai comparisons are on different benchmark suites and are not strictly iso-eval.